Day 5 — Goal 2: Evaluate Chunking Strategies for Production-Style RAG Indexing

> **Guide Version:** 1.0 | **Target Role:** Generative AI Intern | **Difficulty:** Intermediate–Advanced

Stated Assumptions (Read Before Touching Code)

These are not negotiable. If your setup diverges from these, you will need to adapt accordingly — the guide will not silently cover for you.

Assumption	Detail
Corpus location	Plain text or Markdown-like documents (`.txt`, `.md`) stored under `data/corpus/`. Each file is one document. Filenames serve as stable document IDs.
Corpus size	You will index a subset of **50 documents** to control cost and iteration speed. Scale to the full corpus only after validating the pipeline.
Vector search	A **local, in-memory TF-IDF index** built with `scikit-learn`. No external vector database is required. The architecture section explains how to swap this out for FAISS, Chroma, Qdrant, Weaviate, Pinecone, or pgvector.
LLM provider	**Groq API** with model `llama-3.3-70b-versatile`.
Groq is not an embedding model	Groq's chat endpoint cannot generate dense embeddings. It will be used for: chunk ID normalization, metadata generation, evaluation explanation, and optional answerability checks. All vector similarity search is handled by TF-IDF. This is a deliberate, documented design tradeoff — not a hack.
Error handling	**Strict / Fail-fast.** Missing env vars, unreadable files, malformed JSONL rows, and failed API calls all raise explicit exceptions and halt execution. Nothing is silently skipped.
Python version	3.10+
OS	Any POSIX-compatible system (Linux/macOS). Windows users: use WSL2.

Table of Contents

---

1. Project Architecture & Overview

1.1 Why Chunking Matters

A RAG system's retrieval quality is ceiling-bounded by its chunks. The LLM can only answer a question if the correct context lands inside the retrieved chunks. You can have the best model in the world — it doesn't matter if retrieval surfaces irrelevant or incomplete chunks. Chunking is where most production RAG systems fail quietly and expensively.

The core tension is simple:

- **Chunks too small** → Each chunk lacks enough context to be semantically coherent. The retriever finds the right general neighborhood but the answer is split across multiple chunks that never get retrieved together.
- **Chunks too large** → Each chunk contains too much unrelated content. Retrieval scores get diluted. The top-5 results are broad, noisy, and the relevant passage is buried.
- **Wrong boundaries** → Even a well-sized chunk can destroy meaning if it cuts across a sentence, a code block, or a section boundary at the wrong place.

There is no universal optimal chunk size. The right strategy depends on your corpus structure, your query distribution, and your retrieval depth (top-k). That is exactly what this project measures.

1.2 The Three Chunking Strategies

Fixed-Size Chunking

Split the document into non-overlapping windows of exactly N tokens, with an optional overlap of M tokens. Completely ignores document structure, sentence boundaries, or semantic coherence. Fast, simple, and surprisingly competitive on unstructured text where there are no meaningful boundaries to exploit.

Tradeoffs:

- ✅ Simple to implement and reason about
- ✅ Consistent, predictable chunk sizes
- ✅ Works well when documents lack structure (e.g., raw transcripts, logs)
- ❌ Frequently cuts mid-sentence or mid-paragraph
- ❌ Context bleeds across chunk boundaries

Sentence-Aware Chunking

Use a sentence tokenizer (e.g., `nltk` or regex) to detect sentence boundaries first, then greedily accumulate sentences into chunks until a target token count is reached. Optionally add sentence-level

overlap between adjacent chunks.

Tradeoffs:

- ✅ Never cuts mid-sentence; chunks are grammatically complete
- ✅ Better coherence than fixed-size on natural language corpora
- ✅ Low implementation overhead
- ❌ Chunk sizes vary based on sentence length distribution
- ❌ Does not respect topic or section boundaries

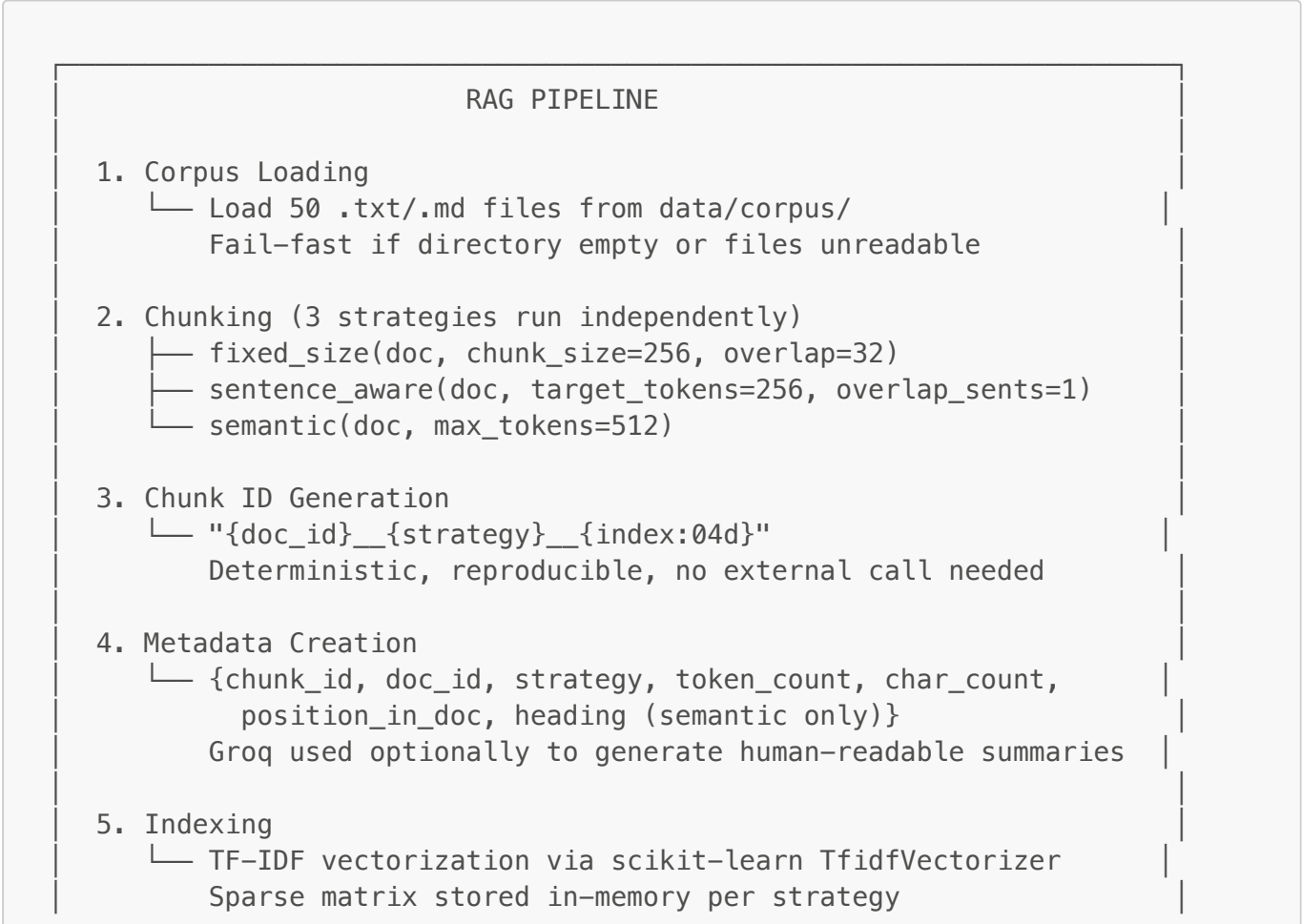
Semantic Chunking

Split on structural or semantic boundaries: headings (Markdown #, ##, ###), blank-line-separated paragraphs, or explicit section markers. Group sections together until a maximum token threshold is reached.

Tradeoffs:

- ✅ Best coherence for structured documents (Markdown, reports, documentation)
- ✅ Each chunk maps to a human-recognizable topic unit
- ✅ Ideal for long documents with clear hierarchical structure
- ❌ Entirely dependent on document quality; unstructured text yields no boundaries
- ❌ Can produce chunks that are too large if sections are verbose
- ❌ Requires knowledge of the document format conventions

1.3 RAG Pipeline Topology



```
│                                                                  │
│  6. Query Vectorization                                          │
│     └── Same TfidfVectorizer.transform(query)                    │
│         Must use the SAME fitted vectorizer as the index         │
│                                                                  │
│  7. Retrieval                                                    │
│     └── cosine_similarity(query_vec, index_matrix)               │
│         Return top-k chunk_ids ranked by score                   │
│                                                                  │
│  8. Recall Scoring                                               │
│     └── recall@k = |retrieved_k ∩ ground_truth| / |ground_truth|  │
│         Measured at k=5 and k=10 for all 25 test questions       │
│                                                                  │
│  9. Winner Selection                                             │
│     └── Aggregate recall@5 and recall@10 per strategy            │
│         Output results.csv + winner.md                           │
└──────────────────────────────────────────────────────────────────┘
```

1.4 Technology Rationale

Choice	Why
Python	Dominant language for ML/AI tooling. Best library ecosystem for text processing, vectorization, and evaluation.
Groq API + `llama-3.3-70b-versatile`	Fast inference, generous free tier, strong instruction following. Used here for metadata generation and explanation, not embedding — important distinction.
scikit-learn TF-IDF	Deterministic, reproducible, zero cost, no external dependency. Correct choice for evaluation benchmarks where you need consistent, explainable retrieval. Do not use it in production for semantic queries — it cannot capture synonyms or paraphrases.
JSONL for test set	One JSON object per line. Streaming-friendly, easy to append or inspect, industry standard for NLP evaluation data.
CSV for results	Tabular evaluation output. Easy to inspect in any spreadsheet tool or pandas.
Markdown for winner.md	Human-readable, version-control-friendly, directly renderable in GitHub/GitLab.

1.5 The TF-IDF / Groq Design Tradeoff (Read This Carefully)

This is worth understanding deeply, not just accepting.

The real-world tradeoff:

TF-IDF is a lexical matching model. It finds chunks that share exact or near-exact vocabulary with the query. It will **fail** on synonyms, paraphrases, and semantic inference. Dense embeddings (e.g., `text-embedding-3-small` from OpenAI, or a local model like `all-MiniLM-L6-v2`) would give better retrieval quality for semantic questions.

However, this project is specifically evaluating **chunking strategy differences**, not embedding model differences. Using TF-IDF as a controlled baseline means any performance gap between strategies is attributable to chunking quality alone, not embedding variation. That is scientifically correct experimental design.

The production implication:

In production, you would replace TF-IDF with dense embeddings + a vector database. The chunk ID scheme, metadata structure, and recall measurement code are all embedding-model-agnostic and can be reused unchanged. Only the vectorization and similarity search steps need swapping. Section 12 covers exactly how to do this.

1.6 Strict / Fail-Fast Error Handling

The pipeline follows a simple contract: **either it runs correctly end-to-end, or it crashes loudly with a message that tells you exactly what is wrong and where.**

There is no "silent fallback," no empty list returned when a file fails to load, no `except: pass`. The reasoning is:

- Silent failures in an evaluation pipeline produce incorrect results that look valid. You end up trusting numbers that are wrong.
- Fail-fast surfaces configuration problems early, before you waste time running a full evaluation on broken input.
- Every error message must be actionable: it must tell you what failed, what the expected value was, and what file/variable to check.

---

2. Repository & Folder Structure

```
chunking_project/
├── chunking/
│   ├── __init__.py              # Package marker
│   ├── strategies.py            # fixed_size, sentence_aware, semantic
chunkers
│   ├── eval.py                  # Full evaluation runner: index →
retrieve → score
│   ├── test_set.jsonl           # 25 questions with ground-truth
chunk_ids
│   ├── results.csv              # Output: recall scores per strategy per
question
│   └── winner.md                # Output: aggregate analysis and
recommendation
│
├── data/
│   └── corpus/                  # Drop your .txt or .md documents here
│       ├── doc_001.md
│       ├── doc_002.md
│       └── ...                  # 50+ documents recommended
│
├── config/
```

```
│   └── settings.py               # Centralized config: paths, chunk sizes,
API params
│
├── utils/
│   ├── __init__.py
│   ├── logging_utils.py          # Structured logger setup
│   ├── groq_client.py            # Thin wrapper around Groq API calls
│   └── io_utils.py               # JSONL read/write, CSV write, corpus
loading
│
├── tests/
│   ├── test_strategies.py        # Unit tests for all three chunkers
│   └── test_eval_metrics.py      # Unit tests for recall@k calculation
│
├── requirements.txt
├── .env                          # Your actual secrets (never commit this)
├── .env.example                  # Template showing required variables
├── output.log                    # Pipeline execution log
└── README.md                     # Quick-start for anyone cloning this
repo
```

---

3. Environment Setup

3.1 Prerequisites

- Python 3.10+
- A Groq API key (free tier at console.groq.com)
- `git`

3.2 Install Dependencies

```
# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate

# Install all dependencies
pip install -r requirements.txt
```

`requirements.txt`:

```
groq>=0.9.0
scikit-learn>=1.4.0
nltk>=3.8.1
python-dotenv>=1.0.0
numpy>=1.26.0
```

After installing, download the NLTK sentence tokenizer data (one-time):

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

3.3 Configure Environment Variables

```
cp .env.example .env
# Edit .env and fill in your GROQ_API_KEY
```

`.env.example`:

```
# Required
GROQ_API_KEY=your_groq_api_key_here

# Optional overrides (defaults shown)
GROQ_MODEL=llama-3.3-70b-versatile
CORPUS_DIR=data/corpus
CHUNK_SIZE_TOKENS=256
OVERLAP_TOKENS=32
MAX_SEMANTIC_TOKENS=512
TOP_K_SMALL=5
TOP_K_LARGE=10
```

3.4 `config/settings.py`

```
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

def _require(key: str) -> str:
    """Fail fast if a required environment variable is missing."""
    value = os.getenv(key)
    if not value:
        raise EnvironmentError(
            f"Required environment variable '{key}' is not set. "
            f"Check your .env file and ensure it contains: {key}=<value>"
        )
    return value


# ── API ──────────────────────────────────────────────────────────────────
GROQ_API_KEY: str = _require("GROQ_API_KEY")
```

```
GROQ_MODEL: str = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")

# ── Paths ──────────────────────────────────────────────────────────────
BASE_DIR = Path(__file__).resolve().parent.parent
CORPUS_DIR = BASE_DIR / os.getenv("CORPUS_DIR", "data/corpus")
TEST_SET_PATH = BASE_DIR / "chunking" / "test_set.jsonl"
RESULTS_PATH = BASE_DIR / "chunking" / "results.csv"
WINNER_PATH = BASE_DIR / "chunking" / "winner.md"
LOG_PATH = BASE_DIR / "output.log"

# ── Chunking Parameters ────────────────────────────────────────────────
CHUNK_SIZE_TOKENS: int = int(os.getenv("CHUNK_SIZE_TOKENS", "256"))
OVERLAP_TOKENS: int = int(os.getenv("OVERLAP_TOKENS", "32"))
MAX_SEMANTIC_TOKENS: int = int(os.getenv("MAX_SEMANTIC_TOKENS", "512"))
OVERLAP_SENTENCES: int = 1

# ── Retrieval ──────────────────────────────────────────────────────────
TOP_K_SMALL: int = int(os.getenv("TOP_K_SMALL", "5"))
TOP_K_LARGE: int = int(os.getenv("TOP_K_LARGE", "10"))
```

---

4. Corpus Preparation

4.1 What Counts as a Valid Corpus Document

Each file in `data/corpus/` must be:

- A `.txt` or `.md` file
- UTF-8 encoded
- At least 200 characters (shorter files are not meaningfully chunkable)
- Named with a stable identifier (the filename becomes the `doc_id`)

4.2 What Makes a Good Corpus for This Experiment

For the experiment to yield meaningful differentiation between strategies, you need variety:

- Some documents with clear Markdown headings (favors semantic chunking)
- Some documents with dense prose and no headings (tests fixed vs. sentence-aware)
- Some documents with mixed structure

If your corpus is all flat prose with no headings, semantic chunking will degrade to paragraph-level chunking and you won't measure its real capability. Choose or generate your 50 documents deliberately.

4.3 `utils/io_utils.py`

```
import json
import csv
import logging
```

```
from pathlib import Path
from typing import Generator

logger = logging.getLogger(__name__)


def load_corpus(corpus_dir: Path, extensions: tuple[str, ...] = (".txt",
".md")) -> dict[str, str]:
    """
    Load all documents from corpus_dir.

    Returns:
        dict mapping doc_id (filename stem) to document text.

    Raises:
        FileNotFoundError: If corpus_dir does not exist.
        ValueError: If corpus_dir contains no valid documents.
        UnicodeDecodeError: If any file is not UTF-8 encoded (fail-fast).
    """
    if not corpus_dir.exists():
        raise FileNotFoundError(
            f"Corpus directory not found: {corpus_dir}\n"
            f"Create it and add .txt or .md documents before running."
        )

    docs: dict[str, str] = {}
    for path in sorted(corpus_dir.iterdir()):
        if path.suffix.lower() not in extensions:
            continue
        text = path.read_text(encoding="utf-8")  # Raises
UnicodeDecodeError on bad files
        if len(text.strip()) < 200:
            logger.warning("Skipping short document (< 200 chars): %s",
path.name)
            continue
        docs[path.stem] = text

    if not docs:
        raise ValueError(
            f"No valid documents found in {corpus_dir}. "
            f"Expected files with extensions: {extensions}"
        )

    logger.info("Loaded %d documents from %s", len(docs), corpus_dir)
    return docs


def iter_jsonl(path: Path) -> Generator[dict, None, None]:
    """
    Yield parsed JSON objects from a JSONL file.

    Raises:
        FileNotFoundError: If path does not exist.
        json.JSONDecodeError: On malformed lines (fail-fast with line
```

```
    number).
    """
    if not path.exists():
        raise FileNotFoundError(f"JSONL file not found: {path}")

    with path.open(encoding="utf-8") as f:
        for line_num, line in enumerate(f, start=1):
            line = line.strip()
            if not line:
                continue
            try:
                yield json.loads(line)
            except json.JSONDecodeError as exc:
                raise json.JSONDecodeError(
                    f"Malformed JSON on line {line_num} of {path}:
{exc.msg}",
                    exc.doc,
                    exc.pos,
                ) from exc


def write_results_csv(path: Path, rows: list[dict]) -> None:
    """Write evaluation results to CSV. Creates parent directories if
needed."""
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows:
        raise ValueError("Cannot write empty results to CSV.")

    fieldnames = list(rows[0].keys())
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    logger.info("Results written to %s (%d rows)", path, len(rows))
```

---

5. Implementation: `chunking/strategies.py`

This is the core deliverable. Each function must:

1. Accept a document string and parameters
2. Return a list of chunk dictionaries with a consistent schema
3. Never silently drop content — every token in the input must appear in exactly one output chunk (for fixed and sentence-aware; semantic may merge short sections)

Chunk Schema

Every chunk returned by any strategy must conform to this schema:

```
{
    "chunk_id": str,        # "{doc_id}__{strategy}__{index:04d}"
    "doc_id": str,          # Source document identifier
    "strategy": str,        # "fixed" | "sentence" | "semantic"
    "index": int,           # 0-based position in document
    "text": str,            # The actual chunk text
    "token_count": int,     # Approximate token count (whitespace split)
    "char_count": int,      # Character count
    "heading": str | None, # Detected heading (semantic only, else None)
}
```

Full `chunking/strategies.py`

```
"""
chunking/strategies.py

Three chunking strategies for RAG indexing evaluation.
All strategies follow the same output schema and use fail-fast error
handling.
"""

from __future__ import annotations

import re
import logging
from typing import Any

import nltk

logger = logging.getLogger(__name__)

# ── Token counting ──────────────────────────────────────────────────────

def _approx_token_count(text: str) -> int:
    """Approximate token count by whitespace splitting. Fast,
consistent."""
    return len(text.split())


# ── Chunk factory ──────────────────────────────────────────────────────

def _make_chunk(
    doc_id: str,
    strategy: str,
    index: int,
    text: str,
    heading: str | None = None,
) -> dict[str, Any]:
```

```
    text = text.strip()
    if not text:
        raise ValueError(
            f"Empty chunk generated by strategy '{strategy}' "
            f"for doc '{doc_id}' at index {index}. "
            "Check your document for long stretches of whitespace."
        )
    return {
        "chunk_id": f"{doc_id}__{strategy}__{index:04d}",
        "doc_id": doc_id,
        "strategy": strategy,
        "index": index,
        "text": text,
        "token_count": _approx_token_count(text),
        "char_count": len(text),
        "heading": heading,
    }


# ── Strategy 1: Fixed-Size ──────────────────────────────────────

def fixed_size(
    text: str,
    doc_id: str,
    chunk_size: int = 256,
    overlap: int = 32,
) -> list[dict[str, Any]]:
    """
    Split document into fixed-size token windows with optional overlap.

    Args:
        text: Full document text.
        doc_id: Stable document identifier.
        chunk_size: Target number of tokens per chunk.
        overlap: Number of tokens to repeat at chunk boundaries.

    Returns:
        List of chunk dicts following the standard schema.

    Raises:
        ValueError: If chunk_size <= 0 or overlap >= chunk_size.
    """
    if chunk_size <= 0:
        raise ValueError(f"chunk_size must be > 0, got {chunk_size}")
    if overlap >= chunk_size:
        raise ValueError(
            f"overlap ({overlap}) must be less than chunk_size
({chunk_size})"
        )

    tokens = text.split()
    if not tokens:
        raise ValueError(f"Document '{doc_id}' produced no tokens after
```

```
splitting.")

    chunks: list[dict[str, Any]] = []
    step = chunk_size - overlap
    start = 0
    index = 0

    while start < len(tokens):
        end = min(start + chunk_size, len(tokens))
        chunk_text = " ".join(tokens[start:end])
        chunks.append(_make_chunk(doc_id, "fixed", index, chunk_text))
        index += 1
        if end == len(tokens):
            break
        start += step

    logger.debug("fixed_size: doc='%s' → %d chunks", doc_id, len(chunks))
    return chunks


# ── Strategy 2: Sentence-Aware ──────────────────────────────────────

def _ensure_nltk_punkt() -> None:
    """Ensure NLTK punkt tokenizer is available. Fail fast if not."""
    try:
        nltk.data.find("tokenizers/punkt_tab")
    except LookupError:
        raise RuntimeError(
            "NLTK 'punkt_tab' tokenizer not found. "
            "Run: python -c \"import nltk; nltk.download('punkt_tab')\""
        )


def sentence_aware(
    text: str,
    doc_id: str,
    target_tokens: int = 256,
    overlap_sentences: int = 1,
) -> list[dict[str, Any]]:
    """
    Group sentences into chunks targeting a token count.
    Never splits mid-sentence. Adds sentence-level overlap at boundaries.

    Args:
        text: Full document text.
        doc_id: Stable document identifier.
        target_tokens: Soft upper bound on tokens per chunk.
        overlap_sentences: Number of trailing sentences from the previous
                           chunk to prepend to the current chunk.

    Returns:
        List of chunk dicts following the standard schema.
```

```
    Raises:
        RuntimeError: If NLTK punkt data is missing.
        ValueError: If document produces no sentences.
    """
    _ensure_nltk_punkt()

    sentences = nltk.sent_tokenize(text)
    if not sentences:
        raise ValueError(f"Document '{doc_id}' produced no sentences after
tokenization.")

    chunks: list[dict[str, Any]] = []
    current_sents: list[str] = []
    current_tokens = 0
    index = 0

    for sent in sentences:
        sent_tokens = _approx_token_count(sent)
        # If adding this sentence would exceed target and we already have
content, flush
        if current_tokens + sent_tokens > target_tokens and current_sents:
            chunk_text = " ".join(current_sents)
            chunks.append(_make_chunk(doc_id, "sentence", index,
chunk_text))
            index += 1
            # Carry over the last N sentences as overlap
            current_sents = current_sents[-overlap_sentences:] if
overlap_sentences > 0 else []
            current_tokens = sum(_approx_token_count(s) for s in
current_sents)

        current_sents.append(sent)
        current_tokens += sent_tokens

    # Flush remainder
    if current_sents:
        chunk_text = " ".join(current_sents)
        chunks.append(_make_chunk(doc_id, "sentence", index, chunk_text))

    if not chunks:
        raise ValueError(
            f"sentence_aware produced 0 chunks for doc '{doc_id}'. "
            "Check document content and target_tokens setting."
        )

    logger.debug("sentence_aware: doc='%s' → %d chunks", doc_id,
len(chunks))
    return chunks


# ── Strategy 3: Semantic
──────────────────────────────────────────────────


# Matches Markdown headings: #, ##, ###, ####
```

```
_HEADING_RE = re.compile(r"^(#{1,4})\s+(.+)$", re.MULTILINE)

# Matches blank lines separating paragraphs
_PARAGRAPH_SPLIT_RE = re.compile(r"\n{2,}")


def _detect_sections(text: str) -> list[dict[str, str | None]]:
    """
    Split document into sections based on Markdown headings.
    Falls back to paragraph splitting if no headings are found.

    Returns:
        List of dicts with keys: 'heading' (str | None), 'body' (str)
    """
    heading_positions = [(m.start(), m.group(0), m.group(2)) for m in
_HEADING_RE.finditer(text)]

    if not heading_positions:
        # No headings: fall back to paragraph splitting
        paragraphs = [p.strip() for p in _PARAGRAPH_SPLIT_RE.split(text) if
p.strip()]
        return [{"heading": None, "body": para} for para in paragraphs]

    sections: list[dict[str, str | None]] = []

    # Content before first heading
    first_pos = heading_positions[0][0]
    preamble = text[:first_pos].strip()
    if preamble:
        sections.append({"heading": None, "body": preamble})

    for i, (pos, heading_line, heading_text) in
enumerate(heading_positions):
        next_pos = heading_positions[i + 1][0] if i + 1 <
len(heading_positions) else len(text)
        # Body is everything after the heading line until next heading
        body_start = pos + len(heading_line)
        body = text[body_start:next_pos].strip()
        sections.append({"heading": heading_text, "body": body})

    return sections


def semantic(
    text: str,
    doc_id: str,
    max_tokens: int = 512,
) -> list[dict[str, Any]]:
    """
    Split document on heading or paragraph boundaries.
    Merges small sections into a single chunk until max_tokens is reached.

    Args:
        text: Full document text.
```

```
        doc_id: Stable document identifier.
        max_tokens: Maximum tokens per chunk. Large sections are split
                    at paragraph boundaries if they exceed this limit.

    Returns:
        List of chunk dicts following the standard schema.

    Raises:
        ValueError: If document produces no sections.
    """
    sections = _detect_sections(text)
    if not sections:
        raise ValueError(
            f"semantic: doc '{doc_id}' produced no sections. "
            "Ensure the document has content."
        )

    chunks: list[dict[str, Any]] = []
    current_parts: list[str] = []
    current_heading: str | None = None
    current_tokens = 0
    index = 0

    for section in sections:
        heading = section["heading"]
        body = section["body"]
        if not body:
            continue

        body_tokens = _approx_token_count(body)

        # If this section alone exceeds max_tokens, flush current buffer
first,
        # then split the large section into sub-chunks
        if body_tokens > max_tokens:
            if current_parts:
                chunk_text = "\n\n".join(current_parts)
                chunks.append(_make_chunk(doc_id, "semantic", index,
chunk_text, current_heading))
                index += 1
                current_parts = []
                current_tokens = 0
                current_heading = None

            # Sub-chunk the oversized section using paragraph splits
            sub_paragraphs = [p.strip() for p in
_PARAGRAPH_SPLIT_RE.split(body) if p.strip()]
            sub_parts: list[str] = []
            sub_tokens = 0
            for para in sub_paragraphs:
                para_tokens = _approx_token_count(para)
                if sub_tokens + para_tokens > max_tokens and sub_parts:
                    chunk_text = "\n\n".join(sub_parts)
                    chunks.append(_make_chunk(doc_id, "semantic", index,
```

```
chunk_text, heading))
                    index += 1
                    sub_parts = []
                    sub_tokens = 0
                sub_parts.append(para)
                sub_tokens += para_tokens
            if sub_parts:
                chunk_text = "\n\n".join(sub_parts)
                chunks.append(_make_chunk(doc_id, "semantic", index,
chunk_text, heading))
                index += 1
            continue

        # If merging this section would exceed max_tokens, flush first
        if current_tokens + body_tokens > max_tokens and current_parts:
            chunk_text = "\n\n".join(current_parts)
            chunks.append(_make_chunk(doc_id, "semantic", index,
chunk_text, current_heading))
            index += 1
            current_parts = []
            current_tokens = 0
            current_heading = None

        # Start tracking the first heading of a new merged group
        if not current_parts and heading:
            current_heading = heading

        current_parts.append(body)
        current_tokens += body_tokens

    # Flush remainder
    if current_parts:
        chunk_text = "\n\n".join(current_parts)
        chunks.append(_make_chunk(doc_id, "semantic", index, chunk_text,
current_heading))

    if not chunks:
        raise ValueError(
            f"semantic produced 0 chunks for doc '{doc_id}'. "
            "Check document content and max_tokens setting."
        )

    logger.debug("semantic: doc='%s' → %d chunks", doc_id, len(chunks))
    return chunks
```

---

6. Implementation: `utils/`

`utils/logging_utils.py`

```
import logging
import sys
from pathlib import Path

from config.settings import LOG_PATH


def setup_logging(level: int = logging.INFO) -> None:
    """Configure root logger to write to both stdout and output.log."""
    LOG_PATH.parent.mkdir(parents=True, exist_ok=True)

    formatter = logging.Formatter(
        fmt="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
        datefmt="%Y-%m-%dT%H:%M:%S",
    )

    handlers: list[logging.Handler] = [
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(LOG_PATH, encoding="utf-8"),
    ]

    root = logging.getLogger()
    root.setLevel(level)
    for h in handlers:
        h.setFormatter(formatter)
        root.addHandler(h)
```

utils/groq_client.py

```
"""
Thin wrapper around the Groq API.
Used for metadata generation and evaluation explanation — NOT for
embeddings.
"""

import logging
from groq import Groq

from config.settings import GROQ_API_KEY, GROQ_MODEL

logger = logging.getLogger(__name__)
_client: Groq | None = None


def get_client() -> Groq:
    global _client
    if _client is None:
        if not GROQ_API_KEY:
            raise EnvironmentError("GROQ_API_KEY is not set. Check your
.env file.")
```

```
        _client = Groq(api_key=GROQ_API_KEY)
    return _client


def chat_complete(system_prompt: str, user_message: str, max_tokens: int =
512) -> str:
    """
    Single-turn chat completion via Groq.

    Raises:
        groq.APIError: On any API-level failure (fail-fast).
    """
    client = get_client()
    response = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ],
        max_tokens=max_tokens,
        temperature=0.0,  # Deterministic output for metadata tasks
    )
    content = response.choices[0].message.content
    if content is None:
        raise ValueError(
            f"Groq returned an empty response for model '{GROQ_MODEL}'. "
            "Check your API quota and the request payload."
        )
    return content.strip()
```

---

7. Building the Test Set: `chunking/test_set.jsonl`

7.1 What the Test Set Is

The test set is 25 questions where you have pre-identified which chunk(s) must appear in the retrieval results to answer the question correctly. This is your ground truth.

Each line in `test_set.jsonl` is a JSON object:

```
{
  "question_id": "q001",
  "question": "What is the default learning rate used in the training
configuration?",
  "ground_truth_chunk_ids": [
    "doc_007__fixed__0012",
    "doc_007__sentence__0008"
  ],
  "notes": "Answer found in doc_007 under 'Training Configuration' section"
}
```

7.2 Important: Ground Truth is Strategy-Specific

Notice that `ground_truth_chunk_ids` contains chunk IDs from multiple strategies. This is correct. For a given question, the "right answer" chunk will have different IDs depending on which strategy produced it. You must:

1. Run each strategy's chunker over all 50 documents first
2. Inspect the resulting chunks and identify which chunks contain the answer
3. Record those chunk IDs per strategy in the test set

The `eval.py` script will look up the appropriate chunk IDs for each strategy when computing recall.

7.3 Test Set Design Principles

- **Cover all 50 documents**: Don't cluster all 25 questions around 3 popular documents. Spread them to detect per-document chunking failures.
- **Vary answer depth**: Include questions whose answers are in: (a) a single sentence, (b) a full paragraph, (c) a multi-section span.
- **Avoid trivially easy questions**: Questions answered by a single unique term will give high recall across all strategies and tell you nothing. Write questions that require retrieving a coherent chunk of moderate length.
- **Include adversarial questions**: 2–3 questions where the answer spans a chunk boundary in fixed-size but not in sentence-aware. These are the most diagnostic questions.

7.4 Generating the Test Set with Groq (Optional Automation)

You can use Groq to propose candidate questions from each document, then manually verify and annotate chunk IDs:

```
from utils.groq_client import chat_complete

SYSTEM = """You are an expert at creating RAG evaluation questions.
Given a document excerpt, generate 3 factual questions that:
1. Have a specific, locatable answer within the excerpt.
2. Cannot be answered from general knowledge alone.
3. Are phrased naturally, as a user would ask them.
Return ONLY a JSON array of question strings. No explanation."""

def propose_questions(doc_text: str, n_chars: int = 1500) -> list[str]:
    """Use Groq to propose candidate questions for a document excerpt."""
    excerpt = doc_text[:n_chars]
    raw = chat_complete(SYSTEM, f"Document excerpt:\n\n{excerpt}")
    import json
    return json.loads(raw)
```

Warning: Groq-generated questions still require manual verification. The model will sometimes generate questions whose answers are not clearly locatable, or that depend on context outside the excerpt. Treat the output as a first draft, not ground truth.

---

8. Implementation: `chunking/eval.py`

This is the evaluation runner. It:

1. Loads all 50 corpus documents
2. Runs all three chunkers on each document
3. Builds a TF-IDF index for each strategy
4. Loads the 25 test questions
5. For each question × strategy, retrieves top-K chunks and computes recall@5 and recall@10
6. Writes `results.csv`
7. Computes aggregate scores and writes `winner.md`

```
"""
chunking/eval.py

Full evaluation pipeline for chunking strategy comparison.
Run: python -m chunking.eval
"""

from __future__ import annotations

import csv
import json
import logging
from pathlib import Path
from typing import Any

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from config.settings import (
    CORPUS_DIR,
    TEST_SET_PATH,
    RESULTS_PATH,
    WINNER_PATH,
    CHUNK_SIZE_TOKENS,
    OVERLAP_TOKENS,
    MAX_SEMANTIC_TOKENS,
    OVERLAP_SENTENCES,
    TOP_K_SMALL,
    TOP_K_LARGE,
)
from chunking.strategies import fixed_size, sentence_aware, semantic
from utils.io_utils import load_corpus, iter_jsonl, write_results_csv
from utils.logging_utils import setup_logging
from utils.groq_client import chat_complete

setup_logging()
logger = logging.getLogger(__name__)

STRATEGIES = {
```

```
    "fixed": lambda text, doc_id: fixed_size(
        text, doc_id, chunk_size=CHUNK_SIZE_TOKENS, overlap=OVERLAP_TOKENS
    ),
    "sentence": lambda text, doc_id: sentence_aware(
        text, doc_id, target_tokens=CHUNK_SIZE_TOKENS,
overlap_sentences=OVERLAP_SENTENCES
    ),
    "semantic": lambda text, doc_id: semantic(
        text, doc_id, max_tokens=MAX_SEMANTIC_TOKENS
    ),
}


# ── Indexing
─────────────────────────────────────────────────────────────────


def build_index(chunks: list[dict[str, Any]]) -> tuple[TfidfVectorizer,
np.ndarray, list[str]]:
    """
    Build a TF-IDF index from a list of chunks.

    Returns:
        (fitted_vectorizer, tfidf_matrix, chunk_id_list)

    Raises:
        ValueError: If chunks list is empty.
    """
    if not chunks:
        raise ValueError("Cannot build index from empty chunk list.")

    texts = [c["text"] for c in chunks]
    chunk_ids = [c["chunk_id"] for c in chunks]

    vectorizer = TfidfVectorizer(
        ngram_range=(1, 2),
        min_df=1,
        max_features=50_000,
        sublinear_tf=True,
    )
    matrix = vectorizer.fit_transform(texts)
    logger.info("Built TF-IDF index: %d chunks, %d features",
matrix.shape[0], matrix.shape[1])
    return vectorizer, matrix, chunk_ids


# ── Retrieval
─────────────────────────────────────────────────────────────────


def retrieve(
    query: str,
    vectorizer: TfidfVectorizer,
    matrix: np.ndarray,
    chunk_ids: list[str],
    top_k: int,
```

```
) -> list[str]:
    """
    Retrieve top_k chunk IDs for a query using TF-IDF cosine similarity.

    Raises:
        ValueError: If top_k > number of indexed chunks.
    """
    if top_k > len(chunk_ids):
        raise ValueError(
            f"top_k={top_k} exceeds total indexed chunks
({len(chunk_ids)}). "
            "Reduce top_k or index more documents."
        )
    query_vec = vectorizer.transform([query])
    scores = cosine_similarity(query_vec, matrix).flatten()
    top_indices = np.argsort(scores)[::-1][:top_k]
    return [chunk_ids[i] for i in top_indices]


# ── Recall Scoring
─────────────────────────────────────────────────────────────

def recall_at_k(retrieved: list[str], ground_truth: list[str]) -> float:
    """
    Compute recall@k.

    recall@k = |retrieved ∩ ground_truth| / |ground_truth|

    Raises:
        ValueError: If ground_truth is empty.
    """
    if not ground_truth:
        raise ValueError(
            "Ground truth chunk list is empty. "
            "Check test_set.jsonl for rows with empty
'ground_truth_chunk_ids'."
        )
    hits = len(set(retrieved) & set(ground_truth))
    return hits / len(ground_truth)


# ── Winner Analysis
──────────────────────────────────────────────────────────────

def _strategy_key(strategy: str, k: int) -> str:
    return f"{strategy}_recall_{k}"


def compute_aggregate_scores(rows: list[dict]) -> dict[str, dict[str,
float]]:
    """Compute mean recall@5 and recall@10 per strategy."""
    strategies = ["fixed", "sentence", "semantic"]
    ks = [TOP_K_SMALL, TOP_K_LARGE]
    scores: dict[str, dict[str, float]] = {}
```

```
    for strategy in strategies:
        scores[strategy] = {}
        for k in ks:
            key = _strategy_key(strategy, k)
            values = [row[key] for row in rows]
            scores[strategy][f"mean_recall@{k}"] = round(sum(values) /
len(values), 4)

    return scores


def write_winner_md(
    path: Path,
    aggregate: dict[str, dict[str, float]],
    rows: list[dict],
) -> None:
    """Write winner.md with aggregate scores and recommendation."""

    # Determine winner by mean recall@5 (primary) and recall@10
(tiebreaker)
    def sort_key(item: tuple[str, dict]) -> tuple[float, float]:
        strategy, scores = item
        return (scores[f"mean_recall@{TOP_K_SMALL}"],
scores[f"mean_recall@{TOP_K_LARGE}"])

    ranked = sorted(aggregate.items(), key=sort_key, reverse=True)
    winner = ranked[0][0]
    runner_up = ranked[1][0]
    last = ranked[2][0]

    # Use Groq to generate a human-readable explanation
    explanation_prompt = f"""
You are a RAG systems expert. Given these chunking evaluation results,
write a concise
2-paragraph explanation of why '{winner}' won and what this means for
production use.
Be specific. Mention corpus structure as a factor.

Results:
{json.dumps(aggregate, indent=2)}

Return only the 2-paragraph explanation. No headers, no bullet points.
"""
    try:
        explanation = chat_complete(
            system_prompt="You are a precise technical writer specializing
in RAG systems.",
            user_message=explanation_prompt,
        )
    except Exception as exc:
        logger.warning("Groq explanation generation failed: %s. Using
fallback.", exc)
        explanation = (
```

```
            f"The '{winner}' strategy achieved the highest mean recall, "
            "suggesting it produces chunks best aligned with the query
vocabulary in this corpus. "
            "Manual review of low-scoring questions is recommended to
identify systematic failure patterns."
        )

    content = f"""# Chunking Strategy Evaluation: Winner Analysis

## Aggregate Scores

| Strategy | mean_recall@{TOP_K_SMALL} | mean_recall@{TOP_K_LARGE} |
|---|---|---|
"""
    for strategy, scores in ranked:
        marker = " ✅ WINNER" if strategy == winner else ""
        content += (
            f"| `{strategy}`{marker} "
            f"| {scores[f'mean_recall@{TOP_K_SMALL}']:.4f} "
            f"| {scores[f'mean_recall@{TOP_K_LARGE}']:.4f} |\n"
        )

    content += f"""
## Recommended Chunker: `{winner}`

{explanation}

## Rule of Thumb

| Corpus Type | Recommended Strategy | Reasoning |
|---|---|---|
| Markdown / structured docs with headings | `semantic` | Headings define
natural topic boundaries; chunks map to coherent sections |
| Plain prose, articles, transcripts | `sentence` | Sentence boundaries
preserve grammatical coherence without relying on formatting |
| Unstructured text, logs, code, tables | `fixed` | No reliable boundaries
exist; consistent size reduces retrieval variance |
| Mixed corpora | `sentence` | Safest general fallback; degrades gracefully
on both structured and unstructured text |

## Per-Question Breakdown

Questions where the winner failed (recall@{TOP_K_SMALL} = 0.0):

"""
    winner_key = _strategy_key(winner, TOP_K_SMALL)
    failed = [r for r in rows if r[winner_key] == 0.0]
    if failed:
        for row in failed:
            content += f"- `{row['question_id']}`: {row.get('question',
'N/A')}\n"
    else:
        content += f"_None — {winner} achieved non-zero
recall@{TOP_K_SMALL} on all questions._\n"
```

```
    content += f"""
## Methodology Notes

- **Vector search:** TF-IDF with cosine similarity (scikit-learn). Lexical
matching only.
- **Recall formula:** `|retrieved_k ∩ ground_truth| / |ground_truth|`
- **Corpus subset:** 50 documents from `data/corpus/`
- **Test set:** 25 questions with manually verified ground-truth chunk IDs
- **Chunk size target:** {CHUNK_SIZE_TOKENS} tokens (fixed/sentence),
{MAX_SEMANTIC_TOKENS} tokens max (semantic)

> **Production note:** TF-IDF retrieval underestimates semantic chunking's
advantage in production,
> because dense embeddings better capture meaning across paraphrase
boundaries where semantic chunks excel.
> If this experiment is reproduced with dense embeddings (e.g., `all-
MiniLM-L6-v2`), expect
> the gap between semantic and fixed-size to widen.
"""

    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content, encoding="utf-8")
    logger.info("Winner analysis written to %s", path)


# ── Main Runner ──────────────────────────────────────────────────────────

def main() -> None:
    logger.info("=== Chunking Strategy Evaluation Pipeline START ===")

    # 1. Load corpus
    corpus = load_corpus(CORPUS_DIR)
    doc_ids = list(corpus.keys())[:50]  # Limit to 50 for cost control
    logger.info("Using %d documents for evaluation", len(doc_ids))

    # 2. Chunk all documents with all strategies
    all_chunks: dict[str, list[dict[str, Any]]] = {name: [] for name in
STRATEGIES}

    for doc_id in doc_ids:
        text = corpus[doc_id]
        for strategy_name, chunker in STRATEGIES.items():
            chunks = chunker(text, doc_id)
            all_chunks[strategy_name].extend(chunks)

    for name, chunks in all_chunks.items():
        logger.info("Strategy '%s': %d total chunks across %d docs", name,
len(chunks), len(doc_ids))

    # 3. Build TF-IDF index per strategy
    indexes: dict[str, tuple[TfidfVectorizer, np.ndarray, list[str]]] = {}
    for name, chunks in all_chunks.items():
```

```
        logger.info("Building index for strategy '%s'...", name)
        indexes[name] = build_index(chunks)

    # 4. Load test set
    test_questions = list(iter_jsonl(TEST_SET_PATH))
    if len(test_questions) != 25:
        raise ValueError(
            f"Expected exactly 25 test questions, found
{len(test_questions)}. "
            f"Check {TEST_SET_PATH}."
        )
    logger.info("Loaded %d test questions", len(test_questions))

    # 5. Evaluate
    result_rows: list[dict] = []

    for question_obj in test_questions:
        qid = question_obj.get("question_id")
        question = question_obj.get("question")
        gt_chunk_ids: dict[str, list[str]] =
question_obj.get("ground_truth_chunk_ids", {})

        if not qid or not question:
            raise ValueError(
                f"Malformed test question — missing 'question_id' or
'question': {question_obj}"
            )
        if not isinstance(gt_chunk_ids, dict):
            raise ValueError(
                f"Question '{qid}': 'ground_truth_chunk_ids' must be a dict
mapping "
                f"strategy name to list of chunk IDs. Got:
{type(gt_chunk_ids)}"
            )

        row: dict[str, Any] = {"question_id": qid, "question": question}

        for strategy_name, (vectorizer, matrix, chunk_ids) in
indexes.items():
            gt = gt_chunk_ids.get(strategy_name, [])
            if not gt:
                logger.warning(
                    "No ground truth for strategy '%s', question '%s'. "
                    "Recall will be 0.0.", strategy_name, qid
                )

            retrieved_5 = retrieve(question, vectorizer, matrix, chunk_ids,
TOP_K_SMALL)
            retrieved_10 = retrieve(question, vectorizer, matrix,
chunk_ids, TOP_K_LARGE)

            row[_strategy_key(strategy_name, TOP_K_SMALL)] = (
                recall_at_k(retrieved_5, gt) if gt else 0.0
            )
```

```
            row[_strategy_key(strategy_name, TOP_K_LARGE)] = (
                recall_at_k(retrieved_10, gt) if gt else 0.0
            )

        result_rows.append(row)
        logger.info(
            "Scored question '%s': fixed@5=%.2f sentence@5=%.2f
semantic@5=%.2f",
            qid,
            row[_strategy_key("fixed", TOP_K_SMALL)],
            row[_strategy_key("sentence", TOP_K_SMALL)],
            row[_strategy_key("semantic", TOP_K_SMALL)],
        )

    # 6. Write results CSV
    # Drop 'question' column from CSV output; keep only IDs and scores
    csv_rows = [
        {k: v for k, v in r.items() if k != "question"} for r in
result_rows
    ]
    write_results_csv(RESULTS_PATH, csv_rows)

    # 7. Compute aggregate and write winner.md
    aggregate = compute_aggregate_scores(result_rows)
    write_winner_md(WINNER_PATH, aggregate, result_rows)

    logger.info("=== Chunking Strategy Evaluation Pipeline COMPLETE ===")
    logger.info("Results: %s", RESULTS_PATH)
    logger.info("Winner analysis: %s", WINNER_PATH)


if __name__ == "__main__":
    main()
```

> **Note on `ground_truth_chunk_ids` schema:** In `eval.py`, the test set schema uses a dict keyed by strategy name (`{"fixed": [...], "sentence": [...], "semantic": [...]}`). This is more expressive than the simple list shown in Section 7, because the same question may have different relevant chunk IDs depending on how the document was chunked. Update your `test_set.jsonl` accordingly.

---

9. Running the Full Pipeline

Step 1: Prepare the corpus

```
# Ensure you have at least 50 documents
ls data/corpus/ | wc -l
```

Step 2: Generate initial chunks and inspect them (do this before building the test set)

```
# Quick inspection script — run from project root
from config.settings import CORPUS_DIR, CHUNK_SIZE_TOKENS, OVERLAP_TOKENS
from utils.io_utils import load_corpus
from chunking.strategies import fixed_size, sentence_aware, semantic

corpus = load_corpus(CORPUS_DIR)
doc_id, text = next(iter(corpus.items()))

fixed_chunks = fixed_size(text, doc_id, chunk_size=CHUNK_SIZE_TOKENS,
overlap=OVERLAP_TOKENS)
sent_chunks = sentence_aware(text, doc_id, target_tokens=CHUNK_SIZE_TOKENS)
sem_chunks = semantic(text, doc_id, max_tokens=512)

print(f"Fixed: {len(fixed_chunks)} chunks")
print(f"Sentence: {len(sent_chunks)} chunks")
print(f"Semantic: {len(sem_chunks)} chunks")
print("\n--- First fixed chunk ---")
print(fixed_chunks[0]["text"][:300])
print("\n--- First semantic chunk ---")
print(sem_chunks[0]["text"][:300])
```

Step 3: Build the test set

Manually inspect chunks across documents, formulate 25 questions, record ground-truth chunk IDs per strategy. Write to `chunking/test_set.jsonl`.

Step 4: Run the evaluation

```
python -m chunking.eval
```

Step 5: Review outputs

```
cat chunking/results.csv
cat chunking/winner.md
```

---

10. Interpreting Results & Writing `winner.md`

`winner.md` is generated automatically by `eval.py`. However, you are expected to add your own manual analysis section with observations that go beyond the numbers. Specifically:

- **Which questions had 0 recall across all strategies?** This suggests either a bad ground-truth annotation or a question that is too paraphrase-dependent for TF-IDF.
- **Where did semantic chunking fail?** Likely on documents with no headings. Identify which docs those are.

- **Where did fixed-size outperform sentence-aware?** This would indicate documents where sentence length is very irregular (long run-on sentences that inflate sentence chunks past the target size).
- **What is the token count distribution per strategy?** High variance in semantic chunks is normal but quantify it.

Add these observations as a `## Manual Analysis` section at the bottom of `winner.md`.

---

11. Unit Tests

`tests/test_strategies.py`

```
"""Unit tests for chunking strategies."""
import pytest
from chunking.strategies import fixed_size, sentence_aware, semantic

SAMPLE_DOC = """
# Introduction

This is the introduction to the document. It covers the main concepts.
The concepts are important for understanding the rest of the material.

# Section One

This section covers the first major topic. There are several key points
here.
Each point builds on the previous one. The section ends with a summary.

# Section Two

This section covers the second major topic. It is longer than the first.
It includes multiple paragraphs and detailed explanations.
The content here is specifically designed to test boundary behavior.
""".strip()

DOC_ID = "test_doc"


class TestFixedSize:
    def test_returns_chunks(self):
        chunks = fixed_size(SAMPLE_DOC, DOC_ID, chunk_size=50, overlap=10)
        assert len(chunks) > 0

    def test_chunk_schema(self):
        chunks = fixed_size(SAMPLE_DOC, DOC_ID, chunk_size=50, overlap=10)
        for chunk in chunks:
            assert "chunk_id" in chunk
            assert "text" in chunk
            assert chunk["strategy"] == "fixed"
            assert chunk["token_count"] > 0
```

```
    def test_chunk_ids_are_unique(self):
        chunks = fixed_size(SAMPLE_DOC, DOC_ID, chunk_size=50, overlap=10)
        ids = [c["chunk_id"] for c in chunks]
        assert len(ids) == len(set(ids))

    def test_no_content_loss(self):
        chunks = fixed_size(SAMPLE_DOC, DOC_ID, chunk_size=50, overlap=0)
        all_tokens = set()
        for chunk in chunks:
            all_tokens.update(chunk["text"].split())
        original_tokens = set(SAMPLE_DOC.split())
        assert original_tokens.issubset(all_tokens)

    def test_invalid_overlap_raises(self):
        with pytest.raises(ValueError, match="overlap"):
            fixed_size(SAMPLE_DOC, DOC_ID, chunk_size=50, overlap=60)


class TestSentenceAware:
    def test_returns_chunks(self):
        chunks = sentence_aware(SAMPLE_DOC, DOC_ID, target_tokens=50)
        assert len(chunks) > 0

    def test_chunk_schema(self):
        chunks = sentence_aware(SAMPLE_DOC, DOC_ID, target_tokens=50)
        for chunk in chunks:
            assert chunk["strategy"] == "sentence"

    def test_chunks_end_at_sentence_boundary(self):
        chunks = sentence_aware(SAMPLE_DOC, DOC_ID, target_tokens=40)
        for chunk in chunks:
            text = chunk["text"].strip()
            # Each chunk should end with sentence-terminating punctuation
            # (allowing for edge cases with trailing whitespace)
            assert text[-1] in ".!?\"'", f"Unexpected chunk end:
'{text[-30:]}'"


class TestSemantic:
    def test_returns_chunks(self):
        chunks = semantic(SAMPLE_DOC, DOC_ID, max_tokens=200)
        assert len(chunks) > 0

    def test_headings_captured(self):
        chunks = semantic(SAMPLE_DOC, DOC_ID, max_tokens=200)
        headings = [c["heading"] for c in chunks if c["heading"] is not
None]
        assert len(headings) > 0
        assert "Introduction" in headings or "Section One" in headings

    def test_no_empty_chunks(self):
        chunks = semantic(SAMPLE_DOC, DOC_ID, max_tokens=200)
        for chunk in chunks:
            assert chunk["text"].strip() != ""
```

```
    def test_fallback_on_no_headings(self):
        flat_doc = "This is paragraph one.\n\nThis is paragraph
two.\n\nThis is paragraph three."
        chunks = semantic(flat_doc, DOC_ID, max_tokens=100)
        assert len(chunks) > 0  # Should fall back to paragraph splitting
```

tests/test_eval_metrics.py

```
"""Unit tests for recall@k metric."""
import pytest
from chunking.eval import recall_at_k


def test_perfect_recall():
    assert recall_at_k(["a", "b", "c"], ["a", "b"]) == 1.0


def test_zero_recall():
    assert recall_at_k(["x", "y", "z"], ["a", "b"]) == 0.0


def test_partial_recall():
    assert recall_at_k(["a", "x", "y"], ["a", "b"]) == pytest.approx(0.5)


def test_empty_ground_truth_raises():
    with pytest.raises(ValueError, match="Ground truth"):
        recall_at_k(["a", "b"], [])


def test_more_retrieved_than_ground_truth():
    # Recall can exceed 1.0 only if retrieved duplicates ground truth —
should not happen
    result = recall_at_k(["a", "b", "c", "d", "e"], ["a"])
    assert result == 1.0


def test_order_independent():
    assert recall_at_k(["c", "b", "a"], ["a", "b"]) == recall_at_k(["a",
"b", "c"], ["a", "b"])
```

Run tests:

```
pytest tests/ -v
```

---

12. Extending to a Production Vector Store

The TF-IDF index in this project is a controlled experiment tool. In production, you will swap it for dense embeddings and a vector database. Here is exactly where the changes happen and how minimal they are.

What Stays the Same

- All three chunkers in `strategies.py` — unchanged
- Chunk ID scheme — unchanged
- Metadata schema — unchanged
- `test_set.jsonl` — unchanged
- `recall_at_k()` metric — unchanged
- `results.csv` structure — unchanged

What Changes

Only two functions in `eval.py` need replacing:

Function	TF-IDF version	Production version
`build_index()`	`TfidfVectorizer.fit_transform(texts)`	Embed all chunk texts → upsert to vector DB
`retrieve()`	`cosine_similarity(query_vec, matrix)`	Embed query → vector DB nearest-neighbor search

Swap Targets

Vector Store	Client Library	Notes
FAISS (local)	`faiss-cpu`	Best for single-machine, large-scale, zero infra overhead
Chroma (local/hosted)	`chromadb`	Simple API, good for prototyping, supports metadata filtering
Qdrant (local/hosted)	`qdrant-client`	Production-grade, excellent filtering, Rust-based performance
Weaviate (hosted)	`weaviate-client`	Schema-based, hybrid search (BM25 + dense) built-in
Pinecone (hosted)	`pinecone-client`	Fully managed, serverless option available
pgvector (PostgreSQL)	`psycopg2` + `pgvector`	Best when you already have a Postgres database

Embedding Model Choice

Groq does not provide an embedding API. For production embeddings, use:

- **Local (free):** `sentence-transformers` with `all-MiniLM-L6-v2` or `BAAI/bge-small-en-v1.5`
- **API (paid):** OpenAI `text-embedding-3-small` or Cohere `embed-english-v3.0`

The `all-MiniLM-L6-v2` model is a strong default: 384-dimensional, runs on CPU, fast enough for 50–500 doc corpora without GPU.

---

13. Common Failure Modes & Debugging

Symptom	Likely Cause	Fix
`EnvironmentError: GROQ_API_KEY not set`	`.env` file missing or not loaded	Run `cp .env.example .env` and add your key
`FileNotFoundError: data/corpus/`	Corpus directory not created	`mkdir -p data/corpus/` and add documents
`RuntimeError: NLTK 'punkt_tab' not found`	NLTK data not downloaded	`python -c "import nltk; nltk.download('punkt_tab')"`
`ValueError: Expected 25 test questions, found N`	Incomplete test set	Add remaining questions to `test_set.jsonl`
All recall scores are 0.0	Wrong ground truth chunk IDs	Regenerate chunks, inspect IDs, re-annotate test set
Semantic chunker returns 1 chunk per document	No headings and very short paragraphs	Check corpus format; may need to lower `max_tokens`
`groq.RateLimitError`	Too many API calls too fast	Add `time.sleep(1)` between Groq calls in the loop
TF-IDF recall suspiciously high for all strategies	Test questions share exact vocabulary with chunk text	Redesign questions to be more paraphrase-based
`json.JSONDecodeError` on line N of test_set.jsonl	Trailing comma, unquoted key, or line encoding issue	Validate JSONL with `python -c "import json; [json.loads(l) for l in open('chunking/test_set.jsonl')]"`

Final Checklist

Before submitting your results:

- [ ] All three chunkers implemented in `strategies.py` with correct output schema
- [ ] Unit tests pass: `pytest tests/ -v`
- [ ] 25 test questions in `test_set.jsonl` with ground-truth chunk IDs verified manually
- [ ] `eval.py` runs end-to-end without errors: `python -m chunking.eval`

- [ ] `results.csv` contains 25 rows with all 6 recall columns populated
- [ ] `winner.md` contains aggregate scores, a recommendation, and your manual analysis
- [ ] `.env` is in `.gitignore` (never commit your API key)
- [ ] `output.log` captures the full pipeline run

Guide authored for Day 5, Goal 2 of the Generative AI Internship program.